

AI-Powered Modern UX for Model-Driven Apps

Power CAT

Power CAT | The Intelligent Enterprise - Power Platform & AI for Frontier Firms

AI-Powered Modern UX for Model-Driven Apps

1. Lab overview

1.1 Lab purpose

This lab demonstrates how **Generative Pages** in **Power Platform** enables makers and developers to rapidly design and evolve applications using natural language. You will build a modern business app using AI-assisted page generation, iterative UX refinement, and automated logic, backed by **Dataverse** for secure and scalable data storage.

Throughout the lab, you will use prompts to generate and refine model-driven app pages, understand how prompt intent shapes UX and functionality, and integrate Generative Pages with Dataverse to deliver an end-to-end solution. The lab also covers best practices for prompt design, governance, and extensibility to ensure production-ready apps with built-in security, RBAC, and enterprise scale.

1.2 Why this matters

Generative Pages empowers development teams to rapidly build and iterate on business applications using natural language, eliminating repetitive scaffolding and accelerating delivery. By integrating with **GitHub Copilot**, teams can choose their own best-of-breed AI model for app generation—bringing the flexibility to select the model that best fits their needs, coding standards, and enterprise requirements. This approach fosters innovation, reduces time-to-value, and keeps developers in control of both the toolchain and the outcome. Once generated, you can take your code and integrate it with your own **source control** system, enabling full ALM practices, team collaboration, and versioned deployments.

2. App scenario

Industry / function	Operations
Primary user role	Operations Manager
Problem statement	How can Northwind Traders rapidly modernize their model-driven app experience with rich, interactive pages built from natural language prompts over Dataverse data

Industry / function	Operations
Intelligent outcome	Generative Pages enables operations teams to generate, iterate, and deploy modern UI pages grounded in Dataverse tables — replacing classic forms and views without manual coding or design effort

3. Core concepts

Gen Pages bring together:

- **AI-powered modernization** for replacing classic model-driven forms and views with rich, interactive pages generated from natural language
- **Dataverse** as the native data backbone, with automatic binding to tables, columns, and relationships
- **Managed Platform** for governed deployment across environments with full ALM support

Concept	Why it matters
AI-powered modernization	Gen Pages replaces classic model-driven forms and views with rich, modern experiences — generated from natural language prompts. It reduces design effort while elevating the end-user experience.
Gen Pages skill	The <code>/genpage</code> skill covers the complete page lifecycle in a single session — from validating prerequisites and authenticating via PAC CLI, through generating a React + TypeScript + Fluent UI component, to deploying directly into a model-driven app and verifying in the browser. It eliminates manual scaffolding and wiring, so you can focus on the experience rather than the toolchain.
Dataverse data binding	Pages connect directly to Dataverse tables, columns, and relationships — ensuring data consistency, security trimming, and real-time sync without manual configuration.
ALM / Solution awareness	Gen Pages are solution-aware by default. Every page is packaged within a Dataverse Solution, enabling confident promotion across dev, test, and production environments.

4. Intelligence used in this lab

Intelligence type	Where it's used	Purpose
GitHub Copilot	Visual Studio Code	Provide a code agent to help the developer quickly code their application
<code>/model-apps:genpage</code> skill		

Intelligence type	Where it's used	Purpose
	GitHub Copilot CLI Chat in Visual Studio Code	Generate model-driven app pages from natural language prompts grounded in Dataverse tables
Power Platform skills for GitHub Copilot	GitHub Copilot CLI Chat in Visual Studio Code	Provide Power Platform-aware tools (environment selection, Dataverse metadata, page authoring) to Copilot
Dataverse MCP server	GitHub Copilot CLI Chat	Expose Dataverse table metadata to Copilot so generated pages bind to real tables and columns
Image grounding in GitHub Copilot CLI Chat	GitHub Copilot CLI Chat in Visual Studio Code	Use a reference screenshot to guide layout and visual style of a generated page

5. Documentation & learning resources

- [Generative pages overview \(Microsoft Learn\)](#)
- [Use generative pages with external tools \(Microsoft Learn\)](#)
- [Power Platform skills for GitHub Copilot \(GitHub repo\)](#)
- [GitHub Copilot documentation](#)
- [Visual Studio Code documentation](#)

6. Prerequisites

Before starting this lab, ensure the following are in place:

Environment & Access

- [Review the shared workshop prerequisites.](#)
- You can create apps in a **Power Platform environment** with Dataverse enabled (Dev/Sandbox preferred)
- Confirm that your attendee account has the **Environment Maker** role.
- Ask a Power Platform administrator to confirm that the environment's data policies permit Dataverse.

Solution & Data

- Ask the workshop facilitator or a Power Platform administrator to import and seed the latest Northwind Traders solution before the lab. If you import and seed it yourself, your account needs the **System Administrator** or **System Customizer** security role, or equivalent custom privileges. [Follow the Northwind Traders solution import and sample data instructions.](#)

Tools

- [Visual Studio Code](#)
- [GitHub Copilot](#) — active subscription and the GitHub Copilot extension installed in VS Code
- [Node.js Long-term support \(LTS\) version](#) (includes npm and npx) — required for all skills

- [.NET SDK](#) — required to install and update the PAC CLI via `dotnet tool`
- [Power Platform CLI](#) >= 2.3.1 — required for schema generation and deployment

Install or update PAC CLI:

```
dotnet tool install -g Microsoft.PowerApps.CLI.Tool
```

Or update if already installed:

```
dotnet tool update -g Microsoft.PowerApps.CLI.Tool
```

Plugins

- Install the **Model Apps plugin** for GitHub Copilot (provides the `/genpage` skill):

```
/plugin marketplace add microsoft/power-platform-skills
/plugin install model-apps@power-platform-skills
```

Licensing

- End users that run model-driven apps need a **Power Apps Premium License** or **Dynamics 365 License**

7. Learning outcomes

By the end of this lab, you will be able to:

- Use the `/model - apps : genpage` skill in GitHub Copilot CLI Chat to generate a model-driven app page from natural language.
- Bind a Gen Page to Dataverse tables and relationships in an existing solution (Northwind Traders).
- Publish a generated page into a new or existing model-driven app and verify it in the browser.
- Iterate on a generated page with follow-up prompts to apply modern styling, conditional formatting, and richer expanded views.
- Use a reference image alongside a prompt to ground the agent when generating a complex order-entry page.

8. Modules covered

Module	Description	Value	Est. time
Generate an Orders page with the <code>genpage</code> skill	Use the <code>/model - apps : genpage</code> skill to scaffold a modern Orders page bound to Northwind Dataverse tables, then publish it to a new app.	Go from natural language to a published, working page in minutes — no manual scaffolding.	25 min
Refine the page with style and behavior prompts	Apply a modern visual style, add conditional formatting for high shipping fees, and	Iterate on UX and behavior through focused prompts	25 min

Module	Description	Value	Est. time
	improve the expanded order view with side-by-side tiles and a map.	without writing code by hand.	
Art of the possible — build a New Order Creation page	Generate a complex order-entry page with shipping, payments, and a paginated product catalog gallery using a single prompt and a reference image.	See how rich prompts and image grounding unlock complex layouts in one pass.	25 min

9. Lab instructions

Variations are expected because AI is non-deterministic, and GitHub Copilot's suggestions can also be influenced by the AI model used by GitHub Copilot, the user's authentication context, permissions, and the specific Power Platform environment being accessed.

Module 1: Generate an Orders page with the genpage skill

Use case value	Use natural language to scaffold a model-driven app page from Dataverse tables
Estimated effort	20 minutes
Scenario	Create a modern Orders experience that lets users browse orders and drill into related order details and products.
Objective	Use the <code>/genpage</code> skill in GitHub Copilot to generate, publish, and verify a Gen Page in a model-driven app.

Tasks covered

1. Start GitHub Copilot CLI Chat in your working directory.
2. Run the `/genpage` skill and select your target environment.
3. Describe the page in natural language and confirm the generation plan.
4. Publish the page to a new model-driven app and verify it in the browser.

Step-by-step instructions

1. Open Visual Studio Code in your current working directory for the Gen Page lab and start GitHub Copilot CLI Chat.

```

 GitHub Copilot v1.0.40-2
Describe a task to get started.

Tip: /theme View or set color mode
Copilot uses AI. Check for mistakes.

• 💡 No copilot instructions found. Run /init to generate a copilot-instructions.md file for this project.
• Environment loaded: 1 hook, 49 skills, 4 MCP servers, 6 plugins, 9 agents, Visual Studio Code connected
• GitHub MCP Server: Connected

D:\Src\2026-04-workshop-prep-genpages
> 
/ commands · ? help
Claude Opus 4.6
```

Figure: Start GitHub Copilot CLI Chat from the activity bar.

2. In the chat input, type `/` to bring up the available skills, then run the Gen Page skill:

```

/model-apps:genpage

/model                                Select AI model to use
> /model-apps:genpage                Creates, updates, and deploys Power Apps generative pages for model-driven apps using React v17, TypeScript, and Fluen...
/power-pages:setup-datam...          Creates Dataverse tables, columns, and relationships for a Power Pages site based on a data model proposal. Use when t...

> /model-apps:genpage
/help show help
Claude Opus 4.6
```

Figure: Invoke the `/model-apps:genpage` skill from GitHub Copilot CLI Chat.

3. The skill validates that it has everything it needs. If you have one or more environments listed, select the environment you want to target.

```
• You have multiple auth profiles. The active ones (marked *) point to the BYOC environment (████████.crm.dynamics.com).

I see several environments available:

- BYOC (active) - █████████.crm.dynamics.com

o Asking user Which environment would you like to work with?

Which environment would you like to work with?

> 1. BYOC - █████████.crm.dynamics.com (Currently active)

6. Other (type your answer)

↑↓ to select · Enter to confirm · Esc to cancel
```

Figure: Choose the environment where the page will be created.

4. When prompted, choose **Create a new page**.

```
• Detect configured languages (shell)
  pac model list-languages
  6 lines...

• Working with environment: BYOC (████████.crm.dynamics.com) - Language: English (1033).

Now let's design your page!

o Asking user Describe the page you'd like to build. You can pick one o...

Describe the page you'd like to build. You can pick one of these examples or type your own description.

>1. Build a page showing Account records as a gallery of cards using modern look & feel. All cards should have fixed size and tall enough to fit 4
   lines of titles. Include name, entityimage on the top and, website, email, phone number. Make the component fill 100% of the space. Make the
   gallery scrollable. Use data from the Account table. Make each card clickable to open the Account record in a new window. The target URL should
   be current location path with following query string parameters: pagetype=entityrecord&etn=[entityname]&id=[recordid] where entityname is account
   and id is accountid.
2. Design a vertically scrollable checklist interface for Task records using a clean, flat layout. Each task should be a row with a left-aligned
   checkbox, subject in bold and right-aligned due date and priority. Use neutral tones for background and soft color tags for priority (e.g., red
   for High, gray for Low). Completed tasks should show a strikethrough and reduced opacity. Allow inline editing of due date with a date picker. On
   hover, rows should highlight with another background. Clicking a task opens the Task record in a new window using:
   pagetype=entityrecord&etn=[entityname]&id=[recordid] where entityname is task and id is related record id.
3. Other (type your answer)

↑↓ to select · Enter to confirm · Esc to cancel
```

Figure: Start a brand-new Gen Page.

5. Select **Other** and enter the following description for your page:

```
I want a modern looking page for Orders. Users should be able to view
Orders and their related data. When I select an Order, the record should
expand and show me the related order details and the product associated
with that Order. Use the `nwind_orders`, `nwind_order_details`, and
`nwind_products` tables from the Northwind Traders solution.
```

```
pagetype=entityrecord&etn=[entityname]&id=[recordid] where entityname is task and id is related record id.  
>3. I want a modern looking page for Orders. Users should be able to View Orders and their related data. When I select an Order, the record should  
expand and show me the related order details, and the product associated with that Order. Use the Order, Order Details, Order Product table.  
↑↓ to select · Enter to confirm · Esc to cancel
```

Figure: Describe the Orders page in natural language.

6. When the skill confirms the page details, select **Yes** to proceed.

```
I see you mentioned Order, Order Details, and Order Product tables. Should I pull live data from these Dataverse entities?  
> 1. Yes, use Dataverse entities (Order, Order Details, Order Product)  
2. No, use mock data  
3. Other (type your answer)  
↑↓ to select · Enter to confirm · Esc to cancel
```

Figure: Confirm the page request to continue.

7. The model may suggest additional enhancements for your page. Select the option that best fits your scenario. For this lab, choose **All of the above (Recommended)**.

```
Any specific requirements? For example, styling preferences, features like search/filtering/sorting, accessibility needs, or interaction  
behaviors?  
> 1. Search/filter by order name or status  
2. Sorting by columns (date, amount, status)  
3. Responsive layout for different screen sizes  
4. Color-coded status badges  
5. All of the above (Recommended)  
6. Other (type your answer)  
↑↓ to select · Enter to confirm · Esc to cancel
```

Figure: Accept the recommended set of enhancements.

8. Review the execution plan that the agent presents before it starts writing code. You can refine the plan here, or select **Looks good, proceed!** to continue.

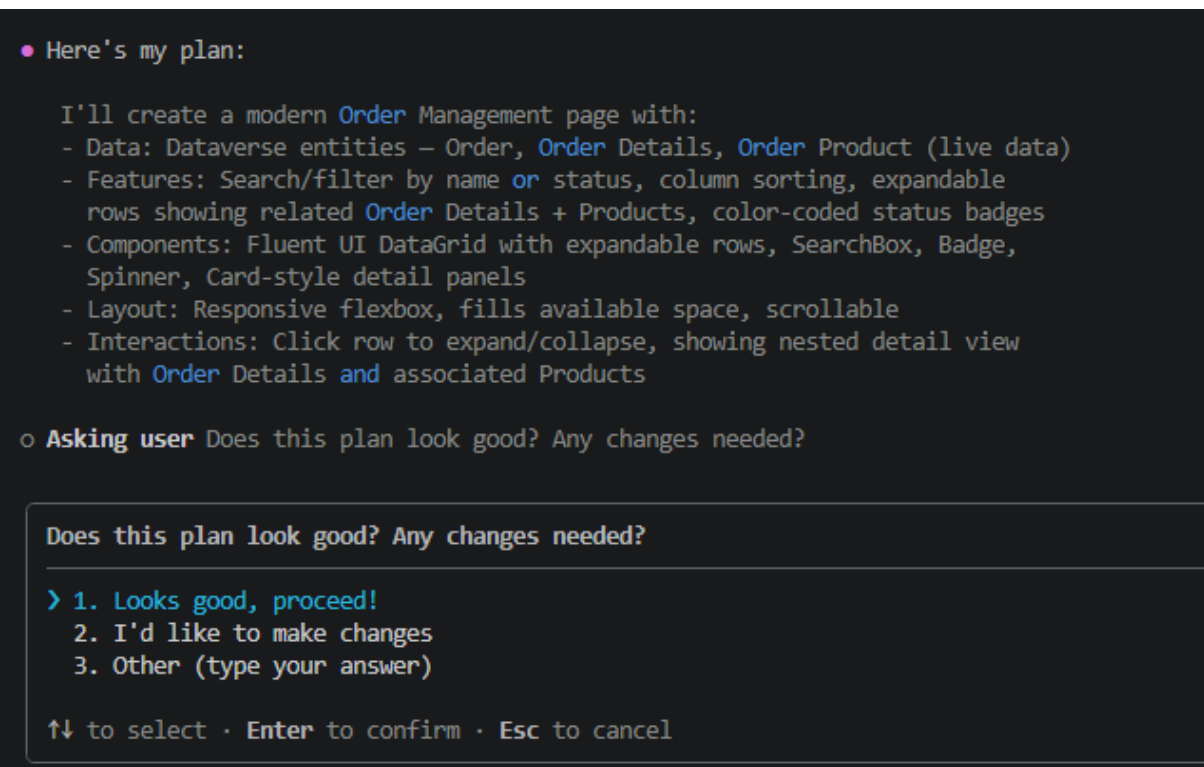


Figure: Confirm the plan the agent will follow.

9. The agent begins creating your page. If you are prompted to allow the skill to read related files, accept the request.

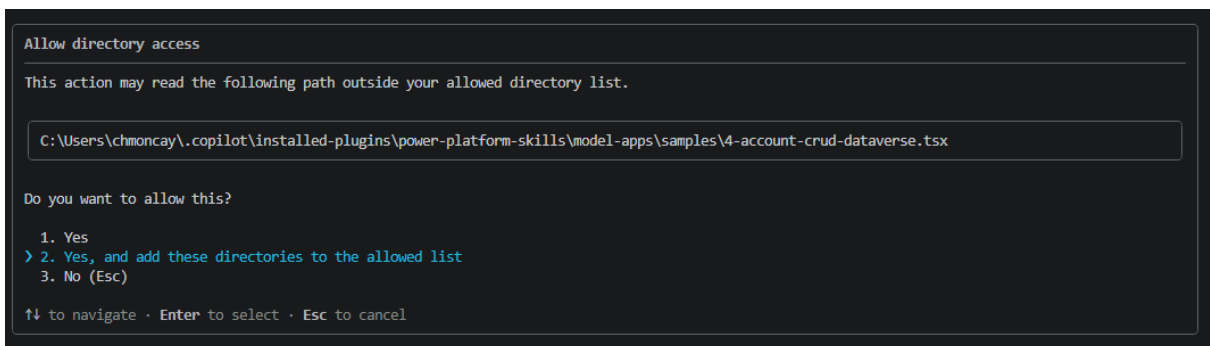


Figure: Approve file reads requested by the skill.

Tip: If Copilot references tables other than the ones you want, remind it which tables to use:

The tables we want are in the Northwind Traders solution with prefix `nwind_`. `nwind_orders`, `nwind_orderdetails`, `nwind_products`

```

X describe_table (MCP: DataverseMcp; ) · tablename: "salesorder"
  ↳ The user rejected this tool call. User feedback: the tables we want are in the Northwind Traders solution with prefix nwind_. nwind_orders, nwind_orderdetails, nwind_products

X describe_table (MCP: DataverseMcp; ) · tablename: "salesorderdetail"
  ↳ The user rejected this tool call. User feedback: the tables we want are in the Northwind Traders solution with prefix nwind_. nwind_orders, nwind_orderdetails, nwind_products

X describe_table (MCP: DataverseMcp; ) · tablename: "product"
  ↳ The user rejected this tool call. User feedback: the tables we want are in the Northwind Traders solution with prefix nwind_. nwind_orders, nwind_orderdetails, nwind_products

• Got it – Northwind Traders tables with nwind_ prefix. Let me get the correct schemas and regenerate the types.

o describe_table (MCP: DataverseMcp; ) · tablename: "nwind_orders"

o describe_table (MCP: DataverseMcp; ) · tablename: "nwind_orderdetails"

o describe_table (MCP: DataverseMcp; ) · tablename: "nwind_products"

```

Figure: Steer the agent to the correct Northwind tables.

10. Once generation completes, the agent prompts you to publish to Power Apps. Select **Publish**.

```

• Good critique. Key adjustments:

1. Master-detail layout instead of expandable rows (DataGrid doesn't support them natively)
2. Use FormattedValue for product names from order details (avoid N+1)
3. Numeric comparisons for status badges + FormattedValue for display text

Now generating the complete code:

• Create order-management.tsx +851

• Code generated! Here's a summary:

Agent Thoughts: Master-detail layout with DataGrid for orders and a slide-up detail panel. Lazy-loads order details + products on row click. Uses FormattedValue annotations for all FK display names and enum labels. Caches orders on window for return visits.

Summary:

- 📊 DataGrid with sortable columns: Order #, Customer, Date, Status, Ship City, Shipping Fee
- 🔍 Search by order number, customer name, or ship name
- 🟢 Color-coded status badges (New=blue, Invoiced=yellow, Shipped=green, Closed=red)
- 📄 Detail panel shows order info + product cards when a row is clicked
- ⚡ Lazy loading of order details and products on expand
- 🌐 Window-level caching with 5-min TTL for return visits
- ♿ Full ARIA labels and keyboard navigation
- 📱 Responsive grid layout for detail cards

o Asking user Would you like to publish this page to Power Apps?

Would you like to publish this page to Power Apps?

> 1. Yes, publish to Power Apps
2. No, I'm done for now
3. Other (type your answer)

↑↓ to select · Enter to confirm · Esc to cancel

```

Figure: Publish the page when generation completes.

11. You are then prompted to choose an app to add the page to.

```

• Here are the available apps. I see a couple relevant ones:

- Northwind Orders (Model-driven) – 2c0ac6c5-67d2-478c-9b8a-9ea95842051b
- Orders Gen Page – 8ff90243-6716-f111-8342-000d3a316172

```

Figure: Pick an existing app or create a new one.

12. Instead of selecting an existing app, ask the agent to create one for you:

Create a new app named Northwind Orders Gen Page

The agent creates the app and adds the page to it.

```
• Check pac model commands (shell)
└─ pac model help
   11 lines...

• Check model create options (shell)
└─ pac model create --help
   9 lines...

• Create new model-driven app (shell)
└─ pac model create --name "Northwind Orders Gen Page" --description "Northwind Traders order management with generative pages" --publish
   7 lines...

❶ Need to specify a solution. Let me check what solutions are available.

• List available solutions (shell)
└─ pac solution list --no-pager 2>&1 | Select-Object -First 30
   6 lines...

• List available solutions (shell)
└─ pac solution list
   16 lines...

• Create app in Northwind solution (shell)
└─ pac model create --name "Northwind Orders Gen Page" --description "Northwind Traders order management with generative pages" --solution
   NorthwindTraders --publish
   6 lines...

• App created! Now deploying the page:

• Upload genpage to Power Apps (shell)
└─ pac model genpage upload `
   --app-id 8847746e-d644-f111-bec6-7ced8d706319 `
   --code-file order-management.tsx `
   --name "Order Management" `
   --data-sources "nwind_orders,nwind_orderdetails,nwind_products" `...

❷ Deploying page to Power Apps (Esc to cancel · 45.6 KiB)
D:\Scr\2026-04-workshop-power-apps
```

Figure: Create a new app named Northwind Orders Gen Page.

13. When the agent finishes, you are prompted to verify the new page.

```
• ☒ Page deployed successfully!

- App: Northwind Orders Gen Page (8847746e-d644-f111-bec6-7ced8d706319)
- Page: Order Management (e3ff0081-01d9-4400-b031-3e85d0780ecf)

o Asking user Would you like to verify the page in the browser using Pl...

Would you like to verify the page in the browser using Playwright? This will open the page and test interactive elements.

> 1. Yes, verify in browser
   2. Skip verification
   3. Other (type your answer)

↑↓ to select · Enter to confirm · Esc to cancel
```

Figure: The agent prompts you to verify the page.

14. Select **Verify in browser**.

```

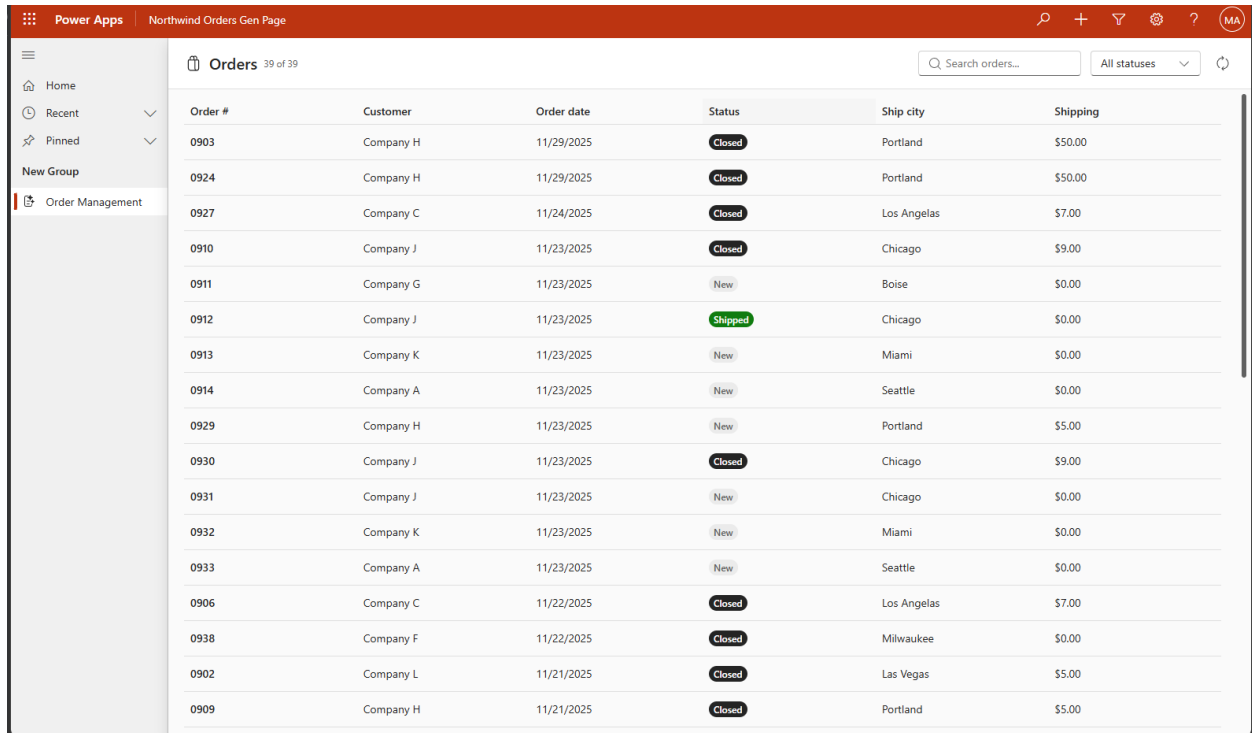

- I can see the sitemap. I need to change the title of the SubArea with GenPageId="95d4b64d-18ae-49f0-b59c-5db81b6b9c5c" from "Create Order" to "New Order Creation".
- update_record (MCP: DataverseMcp) · tablename: "sitemap", recordId: "ddae37e6-2d72-4582-b71d-2b8493c7c433", item: "{\"sitemapxml\":\"<SiteM...
- Publish customizations (shell)
  - pac solution publish
  - 7 lines...
- Done! The sitemap subarea has been renamed from "Create Order" to "New Order Creation" and all customizations are published. The page will now appear with the updated name in the app's navigation.

```

Figure: Open the page in the browser to verify.

Expected result (checkpoint)

The Orders page opens in the browser inside the new **Northwind Orders Gen Page** app and lists records from the Order table.



Order #	Customer	Order date	Status	Ship city	Shipping
0903	Company H	11/29/2025	Closed	Portland	\$50.00
0924	Company H	11/29/2025	Closed	Portland	\$50.00
0927	Company C	11/24/2025	Closed	Los Angeles	\$7.00
0910	Company J	11/23/2025	Closed	Chicago	\$9.00
0911	Company G	11/23/2025	New	Boise	\$0.00
0912	Company J	11/23/2025	Shipped	Chicago	\$0.00
0913	Company K	11/23/2025	New	Miami	\$0.00
0914	Company A	11/23/2025	New	Seattle	\$0.00
0929	Company H	11/23/2025	New	Portland	\$5.00
0930	Company J	11/23/2025	Closed	Chicago	\$9.00
0931	Company J	11/23/2025	New	Chicago	\$0.00
0932	Company K	11/23/2025	New	Miami	\$0.00
0933	Company A	11/23/2025	New	Seattle	\$0.00
0906	Company C	11/22/2025	Closed	Los Angeles	\$7.00
0938	Company F	11/22/2025	Closed	Milwaukee	\$0.00
0902	Company L	11/21/2025	Closed	Las Vegas	\$5.00
0909	Company H	11/21/2025	Closed	Portland	\$5.00

Figure: The generated Orders page running in Power Apps.

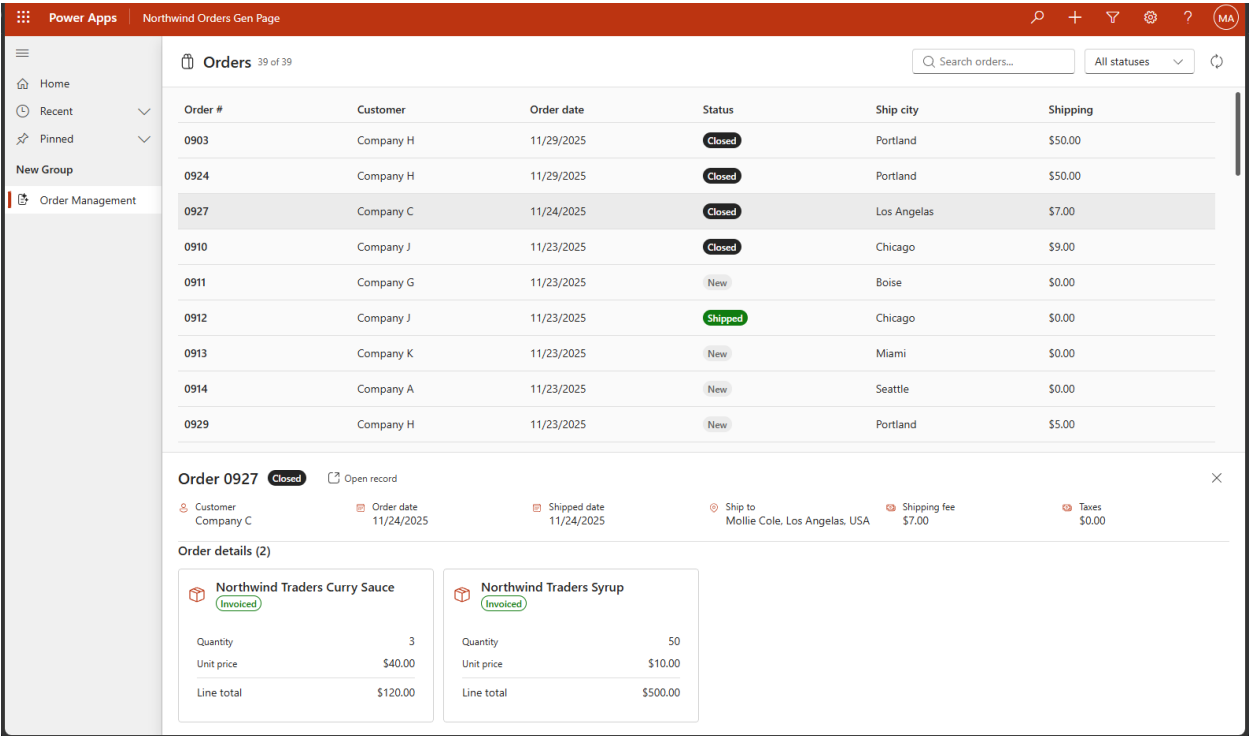


Figure: Browse orders and inspect the generated layout.

Reflection

You generated a working model-driven app page from a single natural language prompt and published it into a new app — no manual scaffolding required.

Module 2: Refine the page with style and behavior prompts

Use case value	Iterate on a generated page using prompts to improve UX, conditional formatting, and the expanded view
Estimated effort	15 minutes
Scenario	Apply a modern visual style, highlight high-value orders, and improve the expanded order details panel.
Objective	Use follow-up prompts to refine layout, styling, conditional formatting, and the expanded record view.

Tasks covered

1. Apply a modern, accessible visual style to the page.
2. Add conditional formatting to highlight orders with high shipping fees.
3. Improve the expanded order details panel with side-by-side tiles and a map.
4. Rename the page in the sitemap.

Step-by-step instructions

1. Back in GitHub Copilot CLI Chat, ask the agent to update the app:

```
What was built:
- Sortable DataGrid with orders (number, customer, date, status, city, shipping)
- Search by order number, customer, or ship name
- Status filter dropdown (New/Invoiced/Shipped/Closed)
- Color-coded status badges
- Click-to-expand detail panel with order info + product cards
- Responsive layout with caching for fast return visits

Want me to make updates or create additional pages?

> Apply a modern, visually expressive style suitable for a model-driven app page, while preserving clarity and accessibility.
Use brighter, saturated colors for page headers and section titles (e.g. blue, purple, teal, or magenta), and lighter tints of those colors for
section backgrounds or highlights.
Keep body content on neutral or softly tinted surfaces to maintain contrast and readability.
Introduce subtle animations for hover, focus, and state changes (short, smooth transitions only; no continuous motion).
Maintain clear contrast ratios for all text and interactive elements, and always preserve visible focus indicators.
Use color to guide attention, not to decorate every element.
Avoid heavy shadows, aggressive gradients, or layout overrides that conflict with the platform.

○ The user wants me to update the existing order-management.tsx page with a modern, visually expressive style. This is an update to an existing
generative page, so I should follow the genpage skill workflow for editing. Let me read the current code, apply the styling changes, and redeploy.

Let me plan the changes:
1. Brighter, saturated colors for headers/titles (blue/purple/teal)
2. Lighter tints for section backgrounds
```

Figure: Continue the chat to make further changes.

2. Enter the following style prompt:

```
Apply a modern, visually expressive style suitable for a model-driven app
page, while preserving clarity and accessibility. Use brighter, saturated
colors for page headers and section titles (e.g. blue, purple, teal, or
magenta), and lighter tints of those colors for section backgrounds or
highlights. Keep body content on neutral or softly tinted surfaces to
maintain contrast and readability. Introduce subtle animations for hover,
focus, and state changes (short, smooth transitions only; no continuous
motion). Maintain clear contrast ratios for all text and interactive
elements, and always preserve visible focus indicators. Use color to guide
attention, not to decorate every element. Avoid heavy shadows, aggressive
gradients, or layout overrides that conflict with the platform.
```

```
• Good, I have the full picture now. Let me apply the modern styling changes. The main edits are:

  1. Header: Vibrant gradient background with white text
  2. Transitions: Smooth hover/focus animations on rows, cards, buttons
  3. Detail panel: Soft tinted background for the info section
  4. Cards: Subtle lift on hover with transition
  5. Info icons: Brighter accent colors
  6. Section titles: Saturated brand colors
  7. Focus indicators: Visible outlines preserved

• Edit order-management.tsx +156 -8

• Now update the sub-components and main render to use the new style classes:

• Edit order-management.tsx +2 -2

• Edit order-management.tsx +2 -2

• Now update the detail panel header and close button:

• Edit order-management.tsx +2 -1

• Now update the "Order details" section title:

• Edit order-management.tsx +1 -1

• Now update the main render - header section with new styled elements:

• Edit order-management.tsx +5 -4

• Update the empty state icon:

• Edit order-management.tsx +1 -1

• Good. Now let me deploy the updated page:
```

Figure: Apply a modern, accessible visual style.

3. The agent applies your changes and republishes the page. Open the page to see the updated visuals.

Power Apps | Northwind Orders Gen Page

Orders 39 of 39

Search orders... All statuses

Order #	Customer	Order date	Status	Ship city	Shipping
0903	Company H	11/29/2025	Closed	Portland	\$50.00
0924	Company H	11/29/2025	Closed	Portland	\$50.00
0927	Company C	11/24/2025	Closed	Los Angeles	\$7.00
0910	Company J	11/23/2025	Closed	Chicago	\$9.00
0911	Company G	11/23/2025	New	Boise	\$0.00
0912	Company J	11/23/2025	Shipped	Chicago	\$0.00
0913	Company K	11/23/2025	New	Miami	\$0.00
0914	Company A	11/23/2025	New	Seattle	\$0.00
0929	Company H	11/23/2025	New	Portland	\$5.00
0930	Company J	11/23/2025	Closed	Chicago	\$9.00
0931	Company J	11/23/2025	New	Chicago	\$0.00
0932	Company K	11/23/2025	New	Miami	\$0.00
0933	Company A	11/23/2025	New	Seattle	\$0.00
0906	Company C	11/22/2025	Closed	Los Angeles	\$7.00
0938	Company F	11/22/2025	Closed	Milwaukee	\$0.00
0902	Company L	11/21/2025	Closed	Las Vegas	\$5.00
0909	Company H	11/21/2025	Closed	Portland	\$5.00

Figure: The Orders page with the new modern style applied.

Power Apps | Northwind Orders Gen Page

Orders 39 of 39

Search orders... All statuses

Order #	Customer	Order date	Status	Ship city	Shipping
0903	Company H	11/29/2025	Closed	Portland	\$50.00
0924	Company H	11/29/2025	Closed	Portland	\$50.00
0927	Company C	11/24/2025	Closed	Los Angeles	\$7.00
0910	Company J	11/23/2025	Closed	Chicago	\$9.00
0911	Company G	11/23/2025	New	Boise	\$0.00
0912	Company J	11/23/2025	Shipped	Chicago	\$0.00
0913	Company K	11/23/2025	New	Miami	\$0.00
0914	Company A	11/23/2025	New	Seattle	\$0.00

Order 0927 Closed [Open record](#)

Customer: Company C | Order date: 11/24/2025 | Shipped date: 11/24/2025 | Ship to: Mollie Cole, Los Angeles, USA | Shipping fee: \$7.00 | Taxes: \$0.00

Order details (2)

Product	Quantity	Unit price	Line total
Northwind Traders Curry Sauce (Invoiced)	3	\$40.00	\$120.00
Northwind Traders Syrup (Invoiced)	50	\$10.00	\$500.00

Figure: Refreshed header and section styling.


```
• ✓ Updated page deployed successfully!

Here's what changed:

- 🌈 Vibrant gradient header – indigo-to-purple with white text and translucent controls
- ✨ Hover transitions – rows get a left-border accent, cards lift with shadow, refresh icon rotates
- 🟡 Color-coded accents – indigo/purple for section titles, icons, and focus rings
- 💜 Tinted info panel – soft lavender background for order details grid
- 📶 Slide-up animation – detail panel entrance with opacity + translate
- 🟦 Visible focus indicators – 2px solid outlines on all interactive elements
- 🟡 Neutral body surfaces – clean white grid area preserves readability

The page is live at:
https://\[redacted\].crm.dynamics.com/main.aspx?appid=8847746e-d644-f111-bec6-7ced8d706319&pagetype=genux&id=e3ff0081-01d9-4400-b031-3e85d0780ecf

Want me to verify it in the browser or make further adjustments?
```

Figure: Subtle animations and tinted surfaces guide attention.

4. Add a conditional formatting prompt to highlight high-value shipping records:

Highlight the records where shipping fee is > \$50.

```
• Deploy page with shipping highlight (shell)
pac model genpage upload `
  --app-id 8847746e-d644-f111-bec6-7ced8d706319 `
  --page-id e3ff0081-01d9-4400-b031-3e85d0780ecf `
  --code-file order-management.tsx `
  --data-sources "nwind_orders,nwind_orderdetails,nwind_products" `...
18 lines...

• ✓ Deployed! Orders with shipping fee > $50 now stand out with a warm amber background and orange left border. The highlight deepens on hover.
Selected-row purple styling still takes priority when a row is clicked.
```

Figure: Conditional formatting highlights high shipping fees.

5. Improve the expanded order details panel with the following prompt:

When a user expands an order, render an expanded view that includes detailed order information, shipping details, and the complete shipping address. Display order and product information within a single, structured tile. Position this tile horizontally adjacent to the shipping details tile to enable clear side-by-side comparison.

Add an additional tile dedicated to visualizing the shipping address on an interactive map. Make sure that map should automatically load and pin the delivery address from the shipping details, centered on that location.

Retain the scroll bar to navigate through the orders gallery.

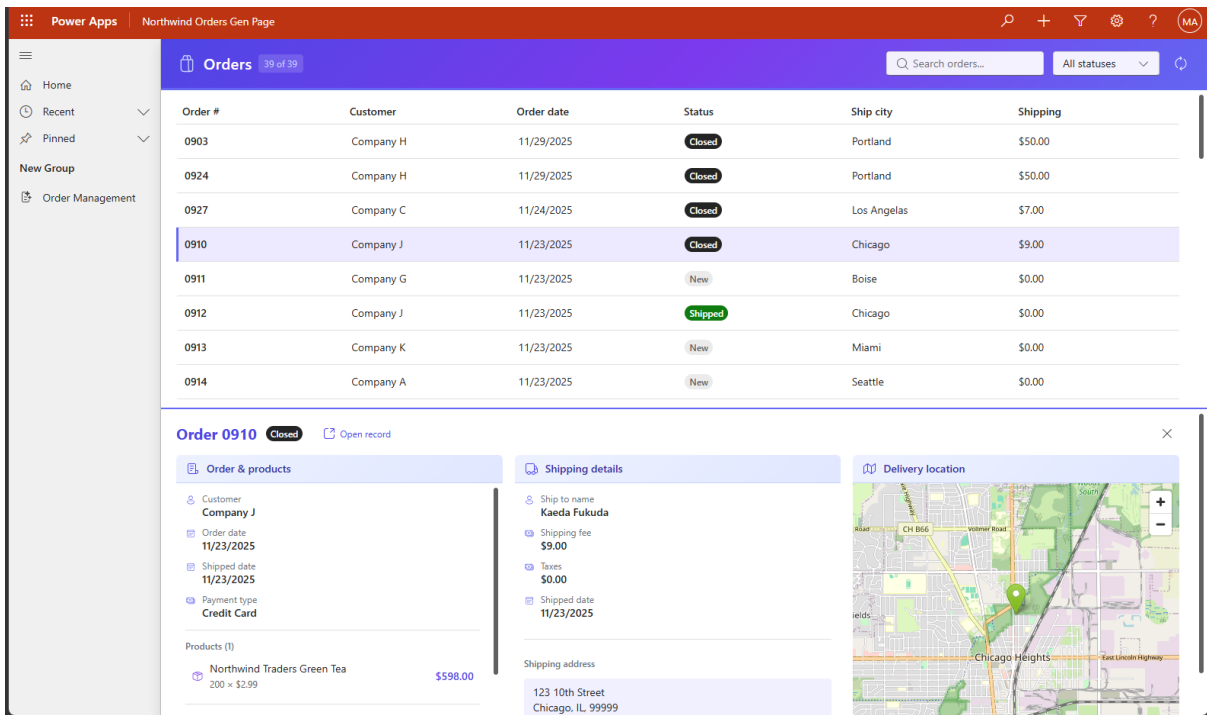


Figure: Expanded view with order, shipping, and a map tile.

6. Rename the page in the sitemap. Edit the subarea directly in the sitemap and set the page name to **Modern Order Page**.

Expected result (checkpoint)

The Orders page now has a modern, accessible visual style, highlights orders with shipping fees over \$50, and shows a richer expanded view with shipping details and an interactive map. The sitemap entry is renamed to **Modern Order Page**.

Reflection

You evolved the page through a sequence of focused prompts — styling, conditional formatting, and an expanded view — without writing any code by hand.

Module 3: Art of the possible — build a New Order Creation page

Use case value	Use a single rich prompt and a reference image to generate a complex order-entry page
Estimated effort	20 minutes
Scenario	Create a new order-entry page with shipping, payments, and a paginated product catalog gallery.
Objective	Generate a complex page with one prompt, then refine it with an image-grounded follow-up.

Tasks covered

1. Generate a new order creation page in the same app.

2. Provide a reference image to ground the agent's design.
3. Add a paginated product catalog gallery with cart actions.
4. Rename the page in the sitemap.

Step-by-step instructions

1. In GitHub Copilot CLI Chat, enter the following prompt to create the new page:

I want to create a new page in the same app which is a modern looking form with animations and styling with modern colorful background. I need to be able to enter all the order details. I also want to see the product catalog to be able to add to the order. Ensure all the columns except Order Number, Status and state code shows up in the form. For each of the columns, show me column name as well. Once the record is created grab the order number and then create a related order detail. Add a separate section to enter all shipping details and another section for payments and taxes. Do not create any mock data. Use the nwind_orders, nwind_order_details, and nwind_products tables from the Northwind Traders solution.

Tip: This page may take a while to create because of its complexity.

The screenshot displays a Power Apps interface for creating a Northwind order. The top navigation bar includes 'Power Apps' and 'Northwind Orders Gen Page'. The main content area is titled 'Create a Northwind order' and features a summary table on the right with the following data:

Field	Value
Order date	2026-04-30
Customer	Not selected
Subtotal	\$0.00
Grand total	\$0.00

The form is organized into three main sections:

- Order Information:** Includes fields for Order date (Thu Apr 30 2026), Customer (Select a Northwind customer), Notes (Add internal notes for order processing), Employee assignment (9 employees available), and Shipper assignment (3 shippers available).
- Product Catalog & Order Lines:** Features a search bar and a grid of product cards. Visible products include Northwind Traders Almonds (\$10.00), Northwind Traders Beer (\$14.00), Northwind Traders Biscuits (\$9.20), and Northwind Traders Brownie Mix (\$12.49). Each card has an 'Add to order' button.
- Shipping Details:** A section at the bottom for defining destination and logistics details.

Figure: The agent generates the order-entry form.

2. You can also provide a sample image for reference. Copy the image below and paste it in the GitHub Copilot CLI chat along with the next prompt.

The image shows a web interface for creating a new order. On the left is a form titled 'Create New Order' with three main sections: 'Order Details', 'Shipping Information', and 'Payment & Tax'. The 'Order Details' section includes fields for Order Number, Order Date, Paid Date, and Shipped Date. The 'Shipping Information' section includes fields for Ship Name, Ship Address, Ship City, Ship State/Province, Ship Zip/Postal Code, and Ship Country. The 'Payment & Tax' section includes fields for Payment Type, Shipping Fee, Tax, and Tax Status. On the right is a 'Product Catalog' section with a 'Cancel' button and a 'Create' button. It displays four product cards, each with a thumbnail image, product name, price, and an 'Add to cart' button. The products are: 'Northwest Tasters Adventure \$10.00', 'Northwest Tasters Beer \$14.00', 'Northwest Tasters Biscuits \$10.00', and 'Northwest Tasters Biscuits Mix \$14.00'.

Figure: Sample reference image for the product catalog layout.

3. Paste the image and send the following prompt:

Display the full product catalog as a vertical gallery positioned on the right side of the order form. Each product entry should be presented as a card that includes a product image as a thumbnail. The gallery should paginate the catalog to show 4 products per page, with clear navigation controls that allow users to move between pages within the catalog. Ensure the layout remains visually aligned with the form and supports smooth browsing without disrupting the order-creation flow.

4. Send one more prompt to add cart interactions:

I want to see a thumbnail of the product image, and ability to add to cart and increase or decrease product quantity

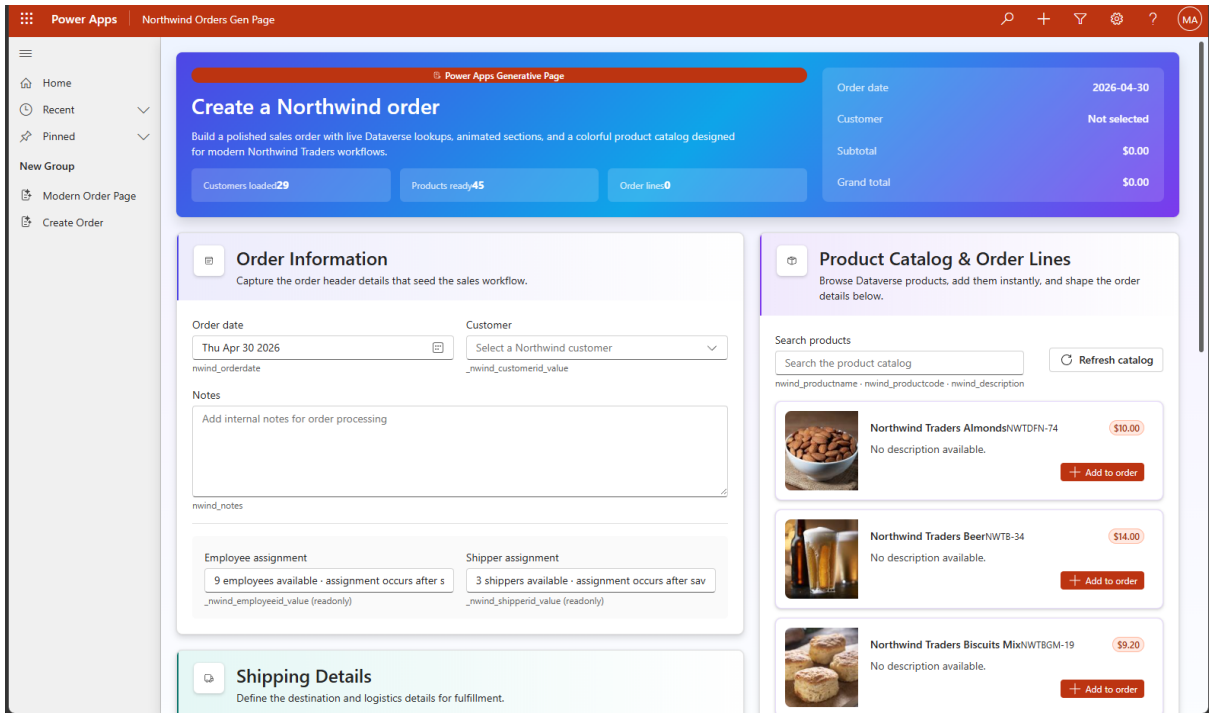


Figure: Paginated product catalog with thumbnails and cart controls.

5. Rename the page in the sitemap. Edit the subarea directly in the sitemap and set the page name to **New Order Creation**.

Expected result (checkpoint)

The app now contains a **New Order Creation** page with sections for order details, shipping, and payments and taxes, alongside a paginated product catalog gallery with add-to-cart and quantity controls.

Reflection

You used a single rich prompt — augmented with a reference image — to generate a complex multi-section page, then refined it iteratively. This is the Art of the Possible with Generative Pages.

11. Lab completion

Congratulations! You used the `/model - apps : genpage` skill to scaffold, refine, and extend a model-driven Orders experience grounded in Dataverse — without writing handcrafted page metadata.

Key takeaways

- You can generate a working model-driven app page from a natural language prompt and publish it to a new app in minutes.
- Iterating with focused prompts lets you restyle pages, add conditional formatting, and reshape the expanded view without leaving GitHub Copilot CLI Chat.
- A single rich prompt — combined with a reference image — can produce a complex multi-section page like the New Order Creation form.

- Grounding Copilot in the Dataverse MCP server ensures generated pages bind to real tables and columns in your environment.
- Generative pages keep you in your developer flow (Visual Studio Code + GitHub Copilot CLI Chat) while still producing first-class model-driven app artifacts.

12. Challenge: apply this to your scenario

Try the genpage skill against your own data and ideas:

- Pick a Dataverse table from your tenant and prompt `/model -apps:genpage` to generate a page for it. What did the model get right? What needed refinement?
- Take a generated page and apply two or three styling/behavior prompts (color theme, conditional formatting, expanded view layout). How few prompts can you use to reach a polished result?
- Capture a screenshot of a page in another product you admire and use it as image grounding for a new page. How close does the generated page get to the reference?
- Identify one form in your existing apps that would benefit from a richer, multi-section layout. Draft a single prompt that could regenerate it end to end.

13. Summary & best practices

Generative pages golden rules:

- **Ground every prompt in real data** — point the genpage skill at the right environment and tables before you generate.
- **Start broad, then refine** — generate the page first, then iterate with small, focused prompts for style, behavior, and layout.
- **Use image grounding for complex layouts** — a reference screenshot is often faster than describing a layout in words.
- **Verify in the browser after each change** — use **Verify in browser** and confirm with **Looks good, proceed!** to keep iterations tight.
- **Keep prompts intent-focused, not metadata-focused** — describe what the user should see and do, and let the skill choose controls and bindings.
- **Treat generated pages as a starting point** — review the produced metadata, rename pages meaningfully, and commit them like any other app artifact.

Troubleshooting

The skill picks the wrong environment

When you start `/model -apps:genpage`, the skill validates that it has everything it needs and asks you to confirm an environment. If your target environment is missing or the wrong one is selected, the generated page will bind to the wrong tables.

- Sign in to Power Platform with the same account in Visual Studio Code that owns your dev environment.
- When the skill lists environments, pick the one that contains the **Northwind Traders** solution (or your own target solution).

- If you picked the wrong environment, cancel the run and re-invoke `/model-apps:genpage` to re-select.

The agent references the wrong tables

The skill may pick up similarly named tables from other solutions instead of the Northwind tables.

Remind the agent which tables to use, including the publisher prefix:

```
The tables we want are in the Northwind Traders solution with prefix nwind_.  
nwind_orders, nwind_orderdetails, nwind_products
```

Copilot prompts to read additional files

While generating a page the agent may ask permission to read skill-related files in your workspace. This is expected — accept the prompts so the skill can complete.

Module 3 takes a long time to generate

The New Order Creation page is intentionally complex (form, shipping, payments, paginated product gallery) and can take several minutes to generate. Let the agent finish before sending follow-up prompts. If you cancel mid-run, you may end up with a partially scaffolded page that needs to be regenerated.

Image grounding doesn't change the layout

If you attached a reference image but the generated page ignores it:

- Confirm the image was attached to the same prompt that describes the layout (not a follow-up message).
- Use a clean, high-contrast screenshot of the target layout — busy or low-resolution images give the model less to work with.
- Be explicit in the prompt about which parts of the image matter (e.g., “use the right-side product gallery layout from the attached image”).

Scripts are blocked from running

If you encounter an error such as:

```
running scripts is disabled on this system
```

Confirm the current execution policy:

```
Get-ExecutionPolicy -List
```

Ensure that `RemoteSigned` is set at either the `LocalMachine` or `CurrentUser` scope.

Running scripts in VS Code or during tool setup

Some development tools (for example, VS Code, npm scripts, or CLI bootstrapping) may encounter script execution errors. First, verify that `RemoteSigned` is set at the `CurrentUser` scope — this resolves most tool setup issues without requiring elevated permissions:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

If the issue persists, you can allow scripts for the current session only using `Bypass` as a last resort:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

- Applies **only to the current PowerShell session**
- Does **not change system-wide settings**
- Resets automatically when the terminal is closed

[!IMPORTANT] `Bypass` removes all warnings and prompts — nothing is blocked. Use it only as a last resort for temporary troubleshooting. Do not use `Bypass` as a persistent or recommended configuration.

Downloaded scripts are blocked

Scripts downloaded from the internet may be blocked even with `RemoteSigned`.

[!IMPORTANT] Before unblocking, inspect the script contents to confirm it comes from a trusted source. Only unblock scripts you have reviewed or that originate from an official repository.

To unblock a script:

```
Unblock-File -Path .\script.ps1
```

Then rerun the script.

Appendix A — Tips for working with the genpage skill

Use this appendix to get the most out of `/model - apps : genpage` and keep your iterations productive.

Tip: Treat each prompt like a focused work item — describe the user outcome, not the metadata. Verify in the browser after each round so problems surface early.

A.1 Writing effective genpage prompts

- **Lead with the user outcome.** Describe what the user should see and do (for example, “view orders and expand a row to see related details”) rather than which controls to add.
- **Name the data explicitly.** Call out the solution and table prefix (for example, “Northwind Traders solution, prefix `nwind_`”) so the skill binds to the right tables.
- **Iterate in small prompts.** Generate the page first, then apply one focused refinement at a time (style, conditional formatting, expanded view, etc.).
- **Use image grounding for complex layouts.** A reference screenshot is faster than describing a layout in words — see Module 3.
- **Always confirm checkpoints.** Use **Verify in browser** after each iteration and **Looks good, proceed!** when the agent shows a plan, so the loop stays tight.

A.2 Common gotchas

- **Wrong environment selected** — re-invoke the skill and pick the environment that contains your target solution.
- **Wrong tables referenced** — remind the agent of the solution name and table prefix.
- **Long-running generations** — Module 3 can take several minutes; don’t cancel mid-run.
- **Partial pages from cancelled runs** — regenerate from scratch rather than patching a half-built page.
- **Permissions or licensing** — make sure your account has maker rights in the target environment and that any required Dataverse tables are present.